Master-Slave Architecture in Jenkins

Introduction

Jenkins **Master-Slave Architecture** is used to **distribute build jobs across multiple machines**, improving performance and efficiency.

- **What is the Master-Slave model in Jenkins?**
- **Why is it important for large-scale CI/CD pipelines?**
- **How can we configure Jenkins to work in a distributed environment?**

This guide explains **how Jenkins Master-Slave architecture works and how to set it up**.

Section 1: Learn

What is Master-Slave Architecture in Jenkins?

In a **Jenkins Master-Slave architecture**:

- **Master Node**:
    - Controls the Jenkins environment.
    - Manages jobs, schedules builds, and monitors nodes.
    - Does not execute heavy builds directly.
- **Slave Nodes (Agents)**:
    - Execute build jobs assigned by the master.
    - Can run on different operating systems.
    - Help in parallel execution and load balancing.

Why Use a Master-Slave Setup?

Without Slaves	**With Slaves**
High CPU and memory usage on a single machine	Distributes workload across multiple machines
Slow execution for multiple jobs	Parallel execution speeds up builds
High risk of failures	Increases system reliability
Limited scalability	Can add more nodes dynamically

Real-Life Example

A **large e-commerce company** needed to run **multiple tests across different platforms (Windows, Linux, macOS). By using a Jenkins Master-Slave setup, they distributed the workload, reducing test execution time from 3 hours to 45 minutes.**

Section 2: Practice

1. Setting Up a Slave Node in Jenkins

Step 1: Configure Master Node

1. **Open Jenkins Dashboard → Click Manage Jenkins → Manage Nodes and Clouds.**
2. **Click New Node → Enter a name (e.g., Slave-1).**
3. **Select "Permanent Agent" → Click OK.**

Step 2: Configure Slave Node

1. **Set Remote Root Directory (e.g., /home/jenkins-slave).**
2. **Choose Launch method:**

- Launch agent via SSH (Recommended for Linux).
- Launch agent via JNLP (For Windows and macOS).

3. Enter the Slave node's IP address.

4. Click Save.

Step 3: Connect the Slave to Jenkins Master

1. Log in to the Slave machine.

2. Run the following command to start the agent:

```
java -jar agent.jar -jnlpUrl
http://master-ip:8080/computer/Slave-1/slave-agent.jnlp -secret
<secret-key> -workDir "/home/jenkins-slave"
```

2. Go back to Jenkins → Verify that the slave node is online.

Why is this useful?

- Ensures distributed execution of Jenkins jobs for better performance.

---

2. Assigning Jobs to Specific Nodes

1. Open Jenkins Dashboard → Click on a Job → Click Configure.

2. Scroll to General Section → Check "Restrict where this project can be run".

3. Enter Label Expression (e.g., Slave-1).

4. Click Save → Build Now.

Why is this important?

- Ensures specific jobs run on the correct machine.

---

3. Verifying Slave Node Performance

To check if the Slave node is processing builds:

1. Click Manage Jenkins → Manage Nodes and Clouds.
2. Click on a Slave Node → View its current status and logs.

To check CPU and memory usage:

```
htop
```

Why is this useful?

- Helps in monitoring and optimizing resource usage on Slave nodes.

Section 3: Know More

Frequently Asked Questions

1. What is the role of the Master in Jenkins?
    - The Master node assigns jobs, manages configurations, and monitors builds.
2. What is the role of a Slave in Jenkins?
    - The Slave node executes build jobs assigned by the Master.
3. How do I check if my Slave is connected?
    - Go to Manage Jenkins → Manage Nodes and check the node status.
4. Can a Slave run multiple jobs?
    - Yes, a single Slave can execute multiple jobs in parallel.
5. What if a Slave node fails?
    - The Master can reassign jobs to other available Slaves.

Conclusion

Jenkins Master-Slave architecture improves performance and scalability.

By configuring Slave nodes, assigning jobs, and monitoring execution, you can efficiently run CI/CD pipelines across multiple machines!